Finished Product: 09/10/2025
Continuing from last time, we kept refining our project until everything was sorted out. There wasn't any need for custom code, as Unity already provided everything we needed — for example, the **XR Reference Image Library**, which housed the 2D sprite images, and the **AR Tracked Image Manager**, which displayed the 3D models above a given picture. Setting this up was extremely simple.

With some spare time to spare, the group decided to take the project a step further by pivoting away from our initial idea of displaying predefined images and instead utilizing AI to create these 3D models in real time.

To accomplish this new goal, we used a service called **Meshy.ai**. They specialize in training AI models to generate 3D objects based on a wide range of parameters — whether it's *text-to-3D* or, as in our case, *image-to-3D*. Luckily, one of our group members had already applied for their student membership, saving us around 17 USD per month, so in total we only had to pay 6 USD.

With a membership established, we gained access to their API, which we could use within our project to send requests in real time and hopefully display the final 3D model above a given picture.

First, we had to create the main script that would orchestrate the whole pipeline — this script was named **GenerateController.cs**, which activates whenever a user clicks a given button. During programming and testing, we ran into issues with our image captures. We noticed that by default, images captured on Android devices were only 640x480, which wasn't ideal for our XR Reference Image Library. To fix this, we created a second script called **CameraCapture.cs**. This class is responsible for overriding the default low resolution and passing the higher-quality image back to **GenerateController.cs**.

Next, using this higher-resolution image, we call another class called **RuntimeImageLibraryController.cs**, which is responsible for storing the image in real time within the XR Reference Image Library for that given session — meaning that if you close the app, it resets. Moving along, we then pass the image to our API with the help of **MeshyClient.cs**. This class sends the request to Meshy to start the generation process.

After the model is generated and textured from our API call, we move on to **GltfLoader.cs**, which downloads and imports the 3D GLB file at runtime using *glTFast* — a library for loading glTF and GLB models into Unity at runtime. This script also sorts the textures and hierarchy placement for the models as live GameObjects. Finally, the last class, **ModelOverlayController.cs**, links the 3D models to the reference images within the XR Reference Image Library and displays the model above its corresponding image.

There are still some issues with getting the logic to work. Currently, when sending the API request, we do receive a response, but the 3D model isn't being displayed above the given image. During debugging, we found that the issue wasn't due to the 3D model being missing at

runtime, but rather a problem somewhere in the code logic. Next time, we'll try to identify and fix the issue, where the culprit could perhaps be either GltfLoader.cs or ModelOverlayController.cs.